Міністерство освіти і науки України

Національний університет "Львівська політехніка"

Кафедра ЕОМ



Звіт

з лабораторної роботи № 5

з дисципліни: «АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ СИСТЕМ»

на тему «Впровадження автоматизованих тестів»

Варіант №21

Виконала:

ст. гр. КІ-404

Хилинська Л. В.

Прийняв:

Федак П. Р

Львів – 2024

Порядок виконання роботи:

1. Implement or use existing test framework
2. Create a set of automated tests;
3. Test report should contain number of all tests, passed tests, failed tests, coverage;
4. Coverage must be more than 80%
5. Required steps

Варіант завдання:

Student number	Game	config format
21	tik-tac-toe 3x3	INI

Теоретичний матеріал

Тестування програмного забезпечення — це процес перевірки програмного продукту для виявлення помилок, забезпечення відповідності вимогам і підвищення якості. Воно включає оцінку функціональності, продуктивності, безпеки та зручності використання.

GTest (Google Test) — це популярний фреймворк для модульного тестування в C++, розроблений Google. Він використовується для автоматизації перевірки роботи окремих компонентів програмного забезпечення, забезпечуючи надійність та якість коду.

Покриття коду (Code Coverage) — це метрика, яка вимірює, яку частину коду було виконано під час тестування. Вона використовується для оцінки якості тестового покриття програми та виявлення областей, які залишилися неперевіреними.

Виконання роботи:

1. Встановлення GTest:

```
cd ~
git clone https://github.com/google/googletest.git
cd googletest
cmake .
make
sudo make install
```

2. Інтеграція GTest з CMake клієнтської частини:

```
#-----------------------------------------
# Test configuration
find_package(GTest REQUIRED)

if (GTest_FOUND)
    add_custom_target(tests
        COMMENT "Building and running tests"
    )

    add_custom_command(TARGET tests PRE_BUILD
        COMMAND ${CMAKE_COMMAND} -E echo "Building test executable"
        COMMAND ${CMAKE_COMMAND} --build ${CMAKE_BINARY_DIR} --target game_tests
    )

    set(TEST_SOURCES
        test/test_main.cpp
        test/test_clientCore.cpp
        test/test_cfgFile.cpp
        # test/test_serialUtils.cpp TBD
        src/serialUtils.cpp
        src/clientCore.cpp
        src/cfgFile.cpp
    )

    add_executable(game_tests EXCLUDE_FROM_ALL ${TEST_SOURCES})

    target_link_libraries(game_tests
        PRIVATE
        gmock
        GTest::GTest
        GTest::Main
        ${SERIALPORT_LIB}
    )

    target_compile_definitions(game_tests PRIVATE SERIAL_PORT_PATH=\"${SERIAL_PORT_PATH}\")

    target_include_directories(game_tests PRIVATE ${CMAKE_SOURCE_DIR}/include)

    if (CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
        target_compile_options(game_tests PRIVATE --coverage)
        target_link_options(game_tests PRIVATE --coverage)
    endif ()

    enable_testing()
    add_test(NAME RunTests COMMAND game_tests)
else ()
    message("GoogleTest needs to be installed to run the tests")
endif ()
```

3. Добавляння тестів у код клієнтської частини (e.g. test_clientCore.cpp):

```
#include <gtest/gtest.h>
#include <gmock/gmock.h>
#include <unordered_map>
#include <cstring>
#include <fstream>

#include "clientCore.h"
#include "serialUtils.h"

struct sp_port *port; //mock
extern const char* CFG_FILE_NAME;

using namespace testing;

TEST(AIMoveTest, WinningMoveSelected) {
    GameState gameState;
    gameState.current_player = 1;
    gameState.player_1 = 'X';
    gameState.player_2 = 'O';

    // Winning move setup
    strcpy(gameState.board, "XX3456789");

    int move = aiMove(&gameState);

    ASSERT_EQ(move, 2); // Winning move at index 2
    ASSERT_EQ(gameState.board[2], 'X'); // 'X' completes the win
}

 TEST(AIMoveTest, BlocksOpponentWinningMove) {
     GameState gameState;
     gameState.current_player = 2;
     gameState.player_1 = 'X';
     gameState.player_2 = 'O';

     // Opponent can win
     strcpy(gameState.board, "XX3456789");

     int move = aiMove(&gameState);

     ASSERT_EQ(move, 2); // AI blocks opponent at index 2
     ASSERT_EQ(gameState.board[2], 'O'); // AI ('O') blocks 'X'
 }

 TEST(HumanMoveTest, ValidMove) {
     GameState gameState;
     gameState.current_player = 1;
     gameState.player_1 = 'X';
     strcpy(gameState.board, "123456789");

     std::istringstream input("1\n"); // Simulate input for position 1
     std::cin.rdbuf(input.rdbuf());

     int move = humanMove(&gameState);

     ASSERT_EQ(move, 0); // First position (0-indexed)
     ASSERT_EQ(gameState.board[0], 'X'); // 'X' marks position
 }
 TEST(DisplayBoardTest, CorrectOutput) {
     char board[BOARD_SIZE] = {'1', '2', '3', '4', '5', '6', '7', '8', '9'};

     testing::internal::CaptureStdout();
     displayBoard(board);
     std::string output = testing::internal::GetCapturedStdout();

     ASSERT_NE(output.find(" 1 | 2 | 3 "), std::string::npos);
     ASSERT_NE(output.find(" 4 | 5 | 6 "), std::string::npos);
     ASSERT_NE(output.find(" 7 | 8 | 9 "), std::string::npos);
```

4. Інтеграція тестів з CMake серверної частини:

```
add_custom_target(test
    COMMAND ${ARDUINO_CLI} compile --fqbn ${BOARD} ${TEST_FILE} --build-property "build.extra_flags=-
Iinclude"
    COMMAND ${ARDUINO_CLI} upload -p ${SERIAL_PORT_PATH} --fqbn ${BOARD} ${TEST_FILE}
    COMMAND script -q /dev/null -c "timeout 5 arduino-cli monitor -p /dev/ttyUSB0"
    WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
    COMMENT "Running tests..."
)
```

5. Добавляння тестів у код серверної частини (e.g. test.ino):

```
#include "../include/serverCore.h"
#include "../src/serverCore.cpp"

int passedTests = 0;
int failedTests = 0;

void test_deserializeCMD_InformNewGame() {
    String message = "100_$";
    int result = (int)deserializeCMD(message);
    if (result == (int)REQUEST_CMD::InformNewGame) {
        passedTests++;
        Serial.println("[V]Test deserializeCMD_InformNewGame Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializeCMD_InformNewGame Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializeCMD_Unknown() {
    String message = "999_$";
    int result = (int)deserializeCMD(message);
    if (result == (int)REQUEST_CMD::Unknown) {
        passedTests++;
        Serial.println("[V]Test deserializeCMD_Unknown Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializeCMD_Unknown Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializePosition_ValidInput() {
    String message = "100_4$";
    int result = deserializePosition(message);
    if (result == 4) {
        passedTests++;
        Serial.println("[V]Test deserializePosition_ValidInput Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializePosition_ValidInput Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializePosition_InvalidInput() {
    String message = "100_$";
    int result = deserializePosition(message);
    if (result == 0) {  // Expect default (or invalid) position
        passedTests++;
        Serial.println("[V]Test deserializePosition_InvalidInput Passed");
    } else {
        failedTests++;
        Serial.print("[X]Test deserializePosition_InvalidInput Failed. Result: ");
        Serial.println(result);
    }
}

void test_deserializeState_CorrectParsing() {
    String message = "100_0_X_O_1_XXXXXXXXX$";
    gamestate_t state;
    deserializeState(message, state);
```

```
if (state.mode == GameMode::ManVsAI &&
    state.player_1 == 'X' &&
    state.player_2 == 'O' &&
    state.current_player == 1 &&
    String(state.board) == String("XXXXXXXXX")) {
    passedTests++;
    Serial.println("[V]Test deserializeState_CorrectParsing Passed");
} else {
    failedTests++;
    Serial.println("[X]Test deserializeState_CorrectParsing Failed");
}
}

void test_checkWin_WinningBoard() {
    char board[10] = "XXX......";
    bool result = checkWin(board);
    if (result) {
        passedTests++;
        Serial.println("[V]Test checkWin_WinningBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkWin_WinningBoard Failed");
    }
}

void test_checkWin_NoWinningBoard() {
    char board[10] = "XXOOOXXOX";
    bool result = checkWin(board);
    if (!result) {
        passedTests++;
        Serial.println("[V]Test checkWin_NoWinningBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkWin_NoWinningBoard Failed");
    }
}

void test_checkDraw_CompleteBoard() {
    gamestate_t state = {GameMode::ManVsMan, 1, "XXOOOXXOX", 'X', 'O'};
    bool result = checkDraw(state);
    if (result) {
        passedTests++;
        Serial.println("[V]Test checkDraw_CompleteBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkDraw_CompleteBoard Failed");
    }
}

}

void test_checkDraw_IncompleteBoard() {
    gamestate_t state = {GameMode::ManVsMan, 1, "XOXOXOX..", 'X', 'O'};
    bool result = checkDraw(state);
    if (!result) {
        passedTests++;
        Serial.println("[V]Test checkDraw_IncompleteBoard Passed");
    } else {
        failedTests++;
        Serial.println("[X]Test checkDraw_IncompleteBoard Failed");
    }
}
```

```
void setup() {
    Serial.begin(9600);
    while (!Serial) {
        ;
    }
    Serial.println("Running tests...");
    delay(100);

    test_deserializeCMD_InformNewGame();
    test_deserializeCMD_Unknown();
    test_deserializePosition_ValidInput();
    test_deserializePosition_InvalidInput();
    test_deserializeState_CorrectParsing();
    test_checkWin_WinningBoard();
    test_checkWin_NoWinningBoard();
    test_checkDraw_CompleteBoard();
    test_checkDraw_IncompleteBoard();

    Serial.print("Tests Passed: ");
    Serial.println(passedTests);
    Serial.print("Tests Failed: ");
    Serial.println(failedTests);

    Serial.println("Test complete.");
}
 void loop() {
     // Nothing needed in loop for now
 }
```

6. У build.sh додано окрему функцію для запуску тестів:

```
run_client_tests() {
    echo "Running client tests..."
    mkdir -p "$CLIENT_DIR/build"

    cd "$CLIENT_DIR/build" || exit

    cmake ..
    cmake --build . --target tests

    ./game_tests

    echo "Generating coverage report..."
    touch cov_output.logs
    #kinda workaround with suppressing error logs
    lcov --capture --directory . --output-file coverage.info --ignore-errors mismatch > cov_output.logs
2>&1
    genhtml coverage.info --output-directory coverage > cov_output.logs 2>&1

    echo "Coverage report generated in: $CLIENT_DIR/build/coverage"

    cd - > /dev/null
    echo "Client tests completed."
}
```

- Додано підтримку опції test:

```
test)
    serial_port=${2:-"/dev/ttyUSB0"} # Default to "/dev/ttyUSB0" if not provided
    run_server_tests "$serial_port"
    run_client_tests
    ;;
```

7. Перевірка результату:

```
lili@lili-VirtualBox      csad2425ki404khvlvnskaliliia21 bash build.sh test
Running server tests...
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/ lili /   csad2425ki404khylynskaliliia21 /serverSide/build
[100%] Running tests...
Sketch uses 6538 bytes (20%) of program storage space. Maximum is 32256 bytes.
Global variables use 1217 bytes (59%) of dynamic memory, leaving 831 bytes for local variables. Maximum
is 2048 bytes.

Used platform Version Path
arduino:avr   1.8.6   /home/ lili /.arduino15/packages/arduino/hardware/avr/1.8.6
New upload port: /dev/ttyUSB0 (serial)
Using generic monitor configuration.
WARNING: Your board may require different settings to work!

Monitor port settings:
  baudrate=9600
  bits=8
  dtr=on
  parity=none
  rts=on
  stop_bits=1

Connecting to /dev/ttyUSB0. Press CTRL-C to exit.
Running tests...

[V]Test deserializeCMD_InformNewRunning tests...
[V]Test deserializeCMD_InformNewGame Passed
[V]Test deserializeCMD_Unknown Passed
[V]Test deserializePosition_ValidInput Passed
[V]Test deserializePosition_InvalidInput Passed
[V]Test deserializeState_CorrectParsing Passed
[V]Test checkWin_WinningBoard Passed
[V]Test checkWin_NoWinningBoard Passed
[V]Test checkDraw_CompleteBoard Passed
[V]Test checkDraw_IncompleteBoard Passed
Tests Passed: 9
Tests Failed: 0
Test complete.
[100%] Built target test
Server tests completed.
Running client tests...
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/ lili /   csad2425ki404khvlvnskaliliia21   /clientSide/build
Building test executable
[100%] Built target game_tests
Built target tests
```

```
[==========] Running 6 tests from 4 test suites.
[----------] Global test environment set-up.
[----------] 2 tests from AIMoveTest
[ RUN      ] AIMoveTest.WinningMoveSelected
AI (Player 1(X)) chose position 3
[       OK ] AIMoveTest.WinningMoveSelected (0 ms)
[ RUN      ] AIMoveTest.BlocksOpponentWinningMove
AI (Player 2(O)) chose position 3
[       OK ] AIMoveTest.BlocksOpponentWinningMove (0 ms)
[----------] 2 tests from AIMoveTest (0 ms total)

[----------] 1 test from HumanMoveTest
[ RUN      ] HumanMoveTest.ValidMove
Player 1(X) enter your move (1-9): [       OK ] HumanMoveTest.ValidMove (0 ms)
[----------] 1 test from HumanMoveTest (0 ms total)

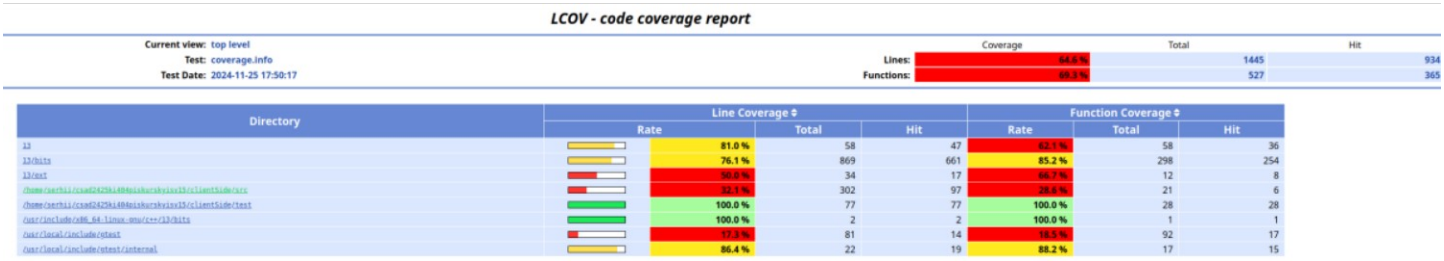
[----------] 1 test from DisplayBoardTest
[ RUN      ] DisplayBoardTest.CorrectOutput
[       OK ] DisplayBoardTest.CorrectOutput (0 ms)
[----------] 1 test from DisplayBoardTest (0 ms total)
```

```
[----------] 1 test from DisplayBoardTest
[ RUN      ] DisplayBoardTest.CorrectOutput
[       OK ] DisplayBoardTest.CorrectOutput (0 ms)
[----------] 1 test from DisplayBoardTest (0 ms total)

[----------] 2 tests from ConfigFileTest
[ RUN      ] ConfigFileTest.ParseValidConfigFile
[       OK ] ConfigFileTest.ParseValidConfigFile (0 ms)
[ RUN      ] ConfigFileTest.WriteConfigFileUpdatesSection
[       OK ] ConfigFileTest.WriteConfigFileUpdatesSection (0 ms)
[----------] 2 tests from ConfigFileTest (0 ms total)

[----------] Global test environment tear-down
[==========] 6 tests from 4 test suites ran. (0 ms total)
[  PASSED  ] 6 tests.
Generating coverage report...
Coverage report generated in: clientSide/build/coverage
Client tests completed.
```

8. Інформація про покриття коду:



LCOV - code coverage report

		Coverage	Total	Hit
Current view:	top level			
Test:	coverage.info	Lines:	1445	934
Test Date:	2024-11-25 17:50:17	Functions:	527	365

Directory	Line Coverage ÷ Rate		Total	Hit	Function Coverage ÷ Rate	Total	Hit
13		81.0 %	58	47	62.1 %	58	36
13/bits		76.1 %	869	661	85.2 %	298	254
13/ext		50.0 %	34	17	66.7 %	12	8
/home/serhii/csad2425si404oiskurskyisv15/clientSide/src		32.1 %	302	97	28.6 %	21	6
/home/serhii/csad2425si404oiskurskyisv15/clientSide/test		100.0 %	77	77	100.0 %	28	28
/usr/include/x86_64-linux-gnu/c++/13/bits		100.0 %	2	2	100.0 %	1	1
/usr/local/include/gtest		17.3 %	81	14	18.5 %	92	17
/usr/local/include/gtest/internal		86.4 %	22	19	88.2 %	17	15

Висновок:Застосування фреймворку GTest у тестуванні програмного забезпечення сприяє ефективному виявленню помилок на початкових етапах розробки, що покращує стабільність і якість коду. Цей інструмент дозволяє створювати автоматизовані модульні тести, які легко інтегруються в процеси CI/CD для безперервного тестування. Використання додаткових інструментів, таких як lcov, для оцінки покриття коду забезпечує детальний аналіз перевірених і неперевірених частин програми. Це дозволяє ретельно протестувати ключові функції та сценарії використання, знижуючи ризики у кінцевому продукті.